CIT CAMPUS PLATFORM

ER Diagram & PostgreSQL Schema Design

Technical design proposal · Version 0.1 · Not frozen

Depends on: Source Audit & Data Dictionary v0.1
Dataset: cit-public-v1
Database: PostgreSQL + PostGIS for navigation

A reviewable relational model for academic data, scheduling, venue bookings, approvals, exams, disruptions, campus navigation, AI audit and competition-compatible imports/exports.

Authority and status

The official starter-pack JSON and validator.py define the competition contract. This is a proposed internal relational model, not a change to the external JSON contract. Do not generate migrations or application code until open decisions are reviewed.

1. Purpose and design principles

• Preserve exact competition import/export formats.

• Separate imported source facts from operational schedules, decisions and user actions.

• Version immutable datasets, timetables and maps.

• Normalize reference arrays into join tables.

• Store solver inputs, version, objective metrics and validator results.

• Keep AI in a proposal role; backend validation and authorised users remain authoritative.

• Use a modular monolith with one PostgreSQL database for the MVP.

2. PostgreSQL schema namespaces

Schema	Responsibility
core	Dataset versions, institute, departments, users, roles, audit
academic	Rooms, faculty, groups, courses, sessions, unavailability
scheduling	Timetable versions, assignments, solver runs, validation
booking	Requests, windows, decisions, allocations, approvals
exams	Exam definitions and allocations
change_mgmt	Disruptions, proposals, approvals, applied changes
navigation	Campus, buildings, floors, spaces, map versions and graph
assistant	AI conversations, messages, tool executions
integration	Imports, exports, source artifacts and validator runs

3. Entity relationship diagrams

Core academic and timetable relationships

```
DATASET_VERSION 1 ■■ * ROOM / FACULTY / GROUP / COURSE / SESSION
DEPARTMENT 1 ■■ * FACULTY / GROUP / COURSE
COURSE 1 ■■ * TEACHING_SESSION
FACULTY 1 ■■ * TEACHING_SESSION
TEACHING_SESSION * ■■ * STUDENT_GROUP (SESSION_GROUP)
TEACHING_SESSION 1 ■■ * SESSION_ALLOWED_ROOM_TYPE
ROOM 1 ■■ * ROOM_FEATURE_ASSIGNMENT * ■■ 1 ROOM_FEATURE
ROOM 1 ■■ * ROOM_UNAVAILABILITY
FACULTY 1 ■■ * FACULTY_UNAVAILABILITY / FACULTY_PREFERENCE
TIMETABLE_VERSION 1 ■■ * TIMETABLE_ASSIGNMENT * ■■ 1 SESSION / ROOM
SOLVER_RUN 1 ■■ * OUTPUT TIMETABLE_VERSION
```

Bookings and approvals

```
BOOKING_REQUEST 1 ■■ * BOOKING_WINDOW
BOOKING_REQUEST 1 ■■ * REQUIRED_FEATURE / ALLOWED_ROOM_TYPE / MANDATORY_GROUP
BOOKING_REQUEST 1 ■■ * CANCELLATION / DECISION
BOOKING_DECISION 1 ■■ * BOOKING_ALLOCATION * ■■ 1 ROOM
BOOKING_REQUEST 1 ■■ * APPROVAL_TASK 1 ■■ * APPROVAL_EVENT
USER_ACCOUNT 1 ■■ * APPROVAL_TASK / APPROVAL_EVENT
```

Exams, disruptions and navigation

```
EXAM * ■■ * COURSE (EXAM_COURSE)
EXAM * ■■ * STUDENT_GROUP (EXAM_GROUP)
EXAM_SCHEDULE_VERSION 1 ■■ * EXAM_ASSIGNMENT 1 ■■ * EXAM_ROOM_ALLOCATION * ■■ ROOM
DISRUPTION_SET 1 ■■ * DISRUPTION_CHANGE
CHANGE_PROPOSAL ■■ BASELINE_TIMETABLE_VERSION + TARGET_DATASET_VERSION
CAMPUS 1 ■■ * BUILDING 1 ■■ * FLOOR 1 ■■ * SPACE
MAP_VERSION 1 ■■ * SPACE / NAVIGATION_NODE / NAVIGATION_EDGE
SPACE may link to canonical ACADEMIC.ROOM
```

4. Conventions

• Internal primary keys use UUID; source IDs are preserved as external IDs unique within dataset version.

• Use timestamptz for source and operational timestamps.

• Teaching periods, event periods and exam sessions are separate time domains.

• Normalize arrays that contain relationships; reserve JSONB for source snapshots, flexible payloads and AI metadata.

• Use ON DELETE RESTRICT for historical master references; avoid cascading deletion of history.

5. Core and calendar tables

Table	Important columns / constraints
core.dataset_versions	id UUID PK; external_dataset_id; version_label; parent_dataset_version_id; source_sha256; status; source_payload JSONB; imported_at; impor
core.institutes	id; code UNIQUE; name; timezone; is_active.
core.departments	id; institute_id FK; code; name; dataset_version_id nullable; external_code. Retain source department codes.
core.calendar_versions	id; dataset_version_id FK; label; fest_week_label; is_active.
core.calendar_days	id; calendar_version_id FK; day_code; ordinal; is_teaching_day. Unique calendar/day.
core.calendar_periods	id; calendar_version_id FK; period_number; start_time; end_time; is_teaching_period. Unique calendar/period.
core.calendar_breaks	id; calendar_version_id FK; break_code; after_period_number; start_time; end_time.
core.calendar_lab_start_periods	calendar_version_id FK; period_number; composite PK.
core.calendar_event_periods	calendar_version_id FK; period_number; composite PK.
core.calendar_exam_days	calendar_version_id FK; day_code; ordinal.
core.calendar_exam_sessions	id; calendar_version_id FK; session_code; start_time; end_time.
booking.priority_categories	id; code UNIQUE; display_name; sort_rank; validator_weight. Do not conflate order rank with validator weight.
booking.role_category_caps	id; dataset_version_id FK; role_code; max_category_id FK; unique dataset/role.

6. Academic master data

Table	Important columns / constraints
academic.rooms	id UUID PK; dataset_version_id FK; external_id; name; building_label; room_type_code; capacity CHECK >=0; accessible; source_payload. Uniqu
academic.room_features	id; code UNIQUE; display_name.
academic.room_feature_assignments	room_id FK; feature_id FK; composite PK.
academic.room_allowed_uses	room_id FK; use_code; composite PK.
academic.faculty	id; dataset_version_id; external_id; name; department_code; optional department_id; max_hours_per_day CHECK >0; needs_accessible; source_
academic.faculty_unavailability	id; faculty_id; day_code; period_number; reason; source_group_key; one row per period.
academic.faculty_preferences	id; faculty_id; preference_type; note; source_payload.
academic.faculty_preference_slots	preference_id; day_code; period_number; composite PK.
academic.student_groups	id; dataset_version_id; external_id; name; year; department_code; size; needs_accessible.
academic.courses	id; dataset_version_id; external_id; title; department_code; year; lecture_hours; tutorial_hours; lab_blocks.
academic.teaching_sessions	id; dataset_version_id; external_id; course_id; faculty_id; kind; size; duration_periods; needs_accessible; basket_slot; fixed_day_code; fixed_start_
academic.session_groups	session_id; student_group_id; composite PK.
academic.session_allowed_room_types	session_id; room_type_code; composite PK.
academic.room_unavailability	id; room_id; day_code; period_number; reason; source_group_key; one row per period.

Important modelling notes

• Keep room building_label as source text until verified campus building records exist; do not invent building identities.

• Accessible room is a dedicated boolean; source `needs` may request `accessible`. Do not duplicate without a deliberate mapping.

• The source faculty and group accessibility flags are not the same as session.needs_accessible; do not silently infer H10 requirements.

• A session needs at least one group and allowed room type; enforce with import/domain validation, not a simple SQL CHECK.

• Expand source unavailability period arrays into individual rows; retain source_group_key if exact reconstruction is required.

7. Scheduling tables

Table	Important columns / constraints
scheduling.timetable_versions	id; dataset_version_id; version_number; status; parent_timetable_version_id; solver_run_id; created_by; approved_by; approved_at. Unique c
scheduling.timetable_assignments	id; timetable_version_id; session_id; day_code; start_period; room_id. Unique timetable/session: at most one assignment per session.
scheduling.assignment_occupancy (optional)	One row per occupied period/resource (room, faculty, group). Unique timetable/resource/day/period prevents overlap if maintained transaction
scheduling.solver_runs	dataset_version_id; input/output timetable versions; solver/version/model version; status; time limit; seed; objective; hard count; soft total; para
scheduling.solver_metrics	solver_run_id; metric_code; metric_value; details. Store S1–S6 and moved-session metrics.
integration.validation_runs	dataset/timetable/export references; validator version; status; hard count; soft total; booking score; priority inversions; report JSON.
integration.validation_violations	validation_run_id; rule_code; severity; entity ID; message; details.

Do not add a simple unique key on room/day/start period: multi-period sessions overlap across multiple periods. Use domain validation and optionally derived occupancy or range constraints. Basket occupancy requires explicit synthetic occupancy-key logic.

8. Booking and approval tables

Table	Important columns / constraints
booking.booking_requests	dataset_version_id; external_id; requester_label; optional requester_user_id; role_code; department; requested/effective category; title; attendance
booking.booking_windows	booking_request_id; day_code; from_period; to_period inclusive; CHECK from <= to.
booking.booking_allowed_room_types	booking_request_id; room_type_code; composite PK.
booking.booking_required_features	booking_request_id; feature_code; composite PK.
booking.booking_mandatory_groups	booking_request_id; student_group_id; composite PK.
booking.booking_cancellations	booking_request_id; cancelled_at; actor label; optional user; note. Cancellation is an event, not deletion.
booking.booking_decisions	booking_request_id; decision_version; status; reason; actor; decided_at; is_effective; export_batch. Partial unique index for one effective decision/
booking.booking_allocations	decision_id; room_id; day; start_period; end_period inclusive; allocation_kind. Current export uses one room per decision.
booking.approval_tasks	request_id; stage; assigned role/user; status; created/due/completed timestamps.
booking.approval_events	task_id; actor; action; reason; occurred_at; metadata. Append-only.

No-self-approval is enforced in the backend service by comparing authenticated requester and actor. The source does not provide a complete reviewer routing matrix; obtain policy or label the routing design as proposed.

9. Examination tables

Table	Important columns / constraints
exams.exams	dataset_version_id; external_id; size; duration_hours; source_payload.
exams.exam_courses	exam_id; course_id; composite PK.
exams.exam_groups	exam_id; student_group_id; composite PK.
exams.exam_schedule_versions	dataset_version_id; version_number; status; parent version; created_at.
exams.exam_assignments	exam_schedule_version_id; exam_id; day_code; exam_session_code. Unique version/exam.
exams.exam_room_allocations	exam_assignment_id; room_id; composite PK. Multiple rooms per exam supported.

10. Disruption and change management

Table	Important columns / constraints
change_mgmt.disruption_sets	source/base dataset; derived dataset; description; status; submitted/approved actors; source payload.
change_mgmt.disruption_changes	disruption_set_id; sequence; change_type; target external ID; typed payload JSONB. Validate payload per type.
change_mgmt.change_proposals	baseline timetable version; target dataset version; proposed timetable; status; objective; moved sessions; hard violations.
change_mgmt.change_proposal_items	proposal; session; change kind; before/after assignment snapshots; explanation.
change_mgmt.change_approvals	proposal; decision; actor; reason; decided_at. Apply only after Admin approval.

Apply changes transactionally: confirm proposal baseline is current or revalidate; activate the derived dataset and timetable versions atomically; preserve the baseline. Permanent disruptions are distinct from Week 7 booking overrides.

11. Users, audit and notifications

Table	Important columns / constraints
core.user_accounts	email citext UNIQUE; display_name; password_hash nullable; auth provider/subject; status; timestamps. Never store plaintext passwords.
core.role_assignments	user_id; role_code; scope_type; scope_id; valid_from/until; granted_by. Official competition roles remain admin, department_coordinator, fa
core.audit_logs	actor; action; entity type/id; timestamp; request ID; before/after JSON; metadata. Append-only.
core.notifications	recipient; event; title/body; resource; read_at; created_at. Create after transaction commit.

Student-facing read access can be a product access policy; do not silently add a fifth competition role.

12. Campus Navigator

Table	Important columns / constraints
navigation.campuses	institute_id; code; name; timezone.
navigation.buildings	campus_id; code; name; outdoor_geometry geometry(Polygon,4326) nullable.
navigation.floors	building_id; floor_code; label; level; coordinate_system; width/height units.
navigation.map_assets	storage URI; media type; checksum; source type; processing status; uploader.
navigation.map_versions	campus; version number; status; parent; created; approved; published.
navigation.spaces	map_version; floor; optional linked_room_id; type; label; local geometry JSON; optional geographic geometry; visibility.
navigation.entrances	map version; building; optional space; outdoor/indoor graph nodes; entrance type.
navigation.navigation_nodes	map version; optional floor/space; node type; local x/y or geographic point.
navigation.navigation_edges	map version; from/to nodes; edge type; distance; accessibility; directionality; geometry.
navigation.map_processing_jobs	asset; status; processor version; result payload; timestamps.
navigation.map_review_items	map version; item type; proposed payload; status; reviewer/note.
navigation.map_publications	map_version UNIQUE; publisher; published_at.

Outdoor geometry uses WGS84. Indoor floor-plan coordinates are local per floor, not latitude/longitude. Connect the graphs only at reviewed entrances. AI-extracted geometry remains a draft until a human reviews it. `spaces.linked_room_id` reuses the canonical academic room when appropriate.

13. AI and integration tables

Table	Purpose
assistant.ai_conversations	User-scoped conversation and context scope.
assistant.ai_messages	Role, text, structured payload and timestamp.
assistant.ai_tool_executions	Tool name, input/output payload, status, validation status and timestamps.
integration.import_jobs	Source file/checksum, dataset version, status and report.
integration.source_artifacts	Original source and generated artifacts with URI/checksum.
integration.export_jobs	Dataset/timetable/exam versions, output type/URI/checksum/status.
integration.validation_runs	Official validator run metadata and JSON report.
integration.validation_violations	Normalized rule-level findings.

14. Constraints and indexes

• Unique external IDs within each dataset version.

• Positive durations; non-negative capacities; valid booking window bounds.

• One assignment per session per timetable version; one effective booking decision per request.

• Unique exam assignment per exam schedule version; unique room per exam allocation.

• Use domain validators for interval conflicts, basket semantics, lunch, labs, availability, capacity, priority, exam half-capacity and approvals.

• Index dataset/external ID, timetable/day/room, session/faculty, booking inbox, room allocations, exam slot, audit entity/time and navigation graph edges.

• Use GiST indexes for PostGIS geometry; add only indexes justified by query patterns.

15. Source import mapping

Source path	Target tables
dataset_id / institute	dataset_versions / institutes
calendar.*	calendar_versions, days, periods, breaks, exam sessions
priority_order / role_category_cap	priority_categories / role_category_caps
rooms[]	rooms, room_feature_assignments, room_allowed_uses
room_unavailability[]	room_unavailability (expanded per period)
faculty[]	faculty, faculty_unavailability, faculty_preferences, preference_slots
student_groups[] / courses[]	student_groups / courses
sessions[]	teaching_sessions, session_groups, session_allowed_room_types
booking_requests[]	booking_requests, windows, room types, features, mandatory groups
cancellations[]	booking_cancellations
exams[]	exams, exam_courses, exam_groups

Import sequence: retain source artifact → parse/validate → create importing dataset version → load configuration → master data → sessions/joins → unavailability/preferences → bookings/children → cancellations → exams/joins → referential/domain checks → mark active or failed → persist import report. Use one transaction; do not create phantom referenced entities.

16. Migration order

• Extensions and namespaces.

• Core dataset, institute, department, calendar and configuration.

• Academic master data and lookup tables.

• Unavailability and preferences.

• Sessions and join tables.

• Timetable, assignments and occupancy.

• Solver and validation.

• Booking requests, child tables, decisions, allocations and approvals.

• Exams.

• Disruption and change management.

• Identity, roles, audit and notifications (dependency-aware).

• Navigator and spatial indexes.

• Assistant and integration artifacts.

• Final constraints, indexes and performance refinements.

17. API and repository implications

Database tables are not API contracts. FastAPI domain services should expose stable resources for datasets, academic records, timetables, bookings, exams, disruptions, campus maps, assistant and exports. Each write must authenticate, authorize, validate schema, execute domain rules, re-check conflicts/version inside a transaction, persist audit atomically and notify after commit.

```
cit-campus-platform/
  apps/web/                     # React + TypeScript + Vite
  services/api/app/
    core/                       # settings, auth, permissions, errors
    modules/                    # datasets, academic, scheduling, bookings,
                                # exams, disruptions, navigation, assistant
    db/                         # SQLAlchemy models and session
  services/api/migrations/      # Alembic
  solver/                       # model, constraints, objectives, repair
  data/source/                  # original starter files; never hand-edit
  data/fixtures/
  exports/                      # generated outputs only
  docs/                         # PRD, data dictionary, DB/API/solver specs
  scripts/                      # import, export, official validator
```

18. Decisions to confirm

ID	Decision	Recommendation / status
DB-01	Backend language	Python/FastAPI subject to team capability — OPEN
DB-02	Database topology	One PostgreSQL DB, modular monolith — PROPOSED
DB-03	Dataset history	Immutable and derived dataset versions — PROPOSED
DB-04	Department master	Normalize codes but preserve source codes — PROPOSED
DB-05	Building normalization	Keep source label until map data verified — PROPOSED
DB-06	Accessibility derivation	Do not infer beyond explicit session requirement — OPEN
DB-07	Reviewer routing	Confirm organiser policy — OPEN
DB-08	Recurring booking	Preserve series_id; defer recurrence semantics — OPEN
DB-09	Multi-room booking	Not in current export contract — OUT OF SCOPE
DB-10	Student role	Read access as product policy, not competition role — PROPOSED
DB-11	Indoor coordinates	Local per-floor coordinate system — PROPOSED
DB-12	Map publishing	Reviewed immutable versions with rollback — PROPOSED

19. Acceptance checklist

■ Every source field has a target column or documented source snapshot.

■ Every reference array has a relational mapping.

■ External IDs and dataset scoping are preserved.

■ Teaching, event and exam time domains are separate.

■ Dataset/timetable/map versioning is explicit.

■ Booking request, decision, allocation and approval are separate.

■ Multi-room exam allocation is represented.

■ Disruptions and derived dataset versions are represented.

■ Navigator reuses canonical room identity.

■ AI cannot write/publish directly.

■ Constraints, indexes, imports and exports are reviewed.

■ Team confirms stack and scope before migrations.

Next deliverable: Milestone 2B — API Specification and Backend Domain Contracts. Define endpoints, payloads, responses, permissions, approval states, validation errors and transaction boundaries. Review this schema before generating migrations.